

Network/TLS Posture Analyzer

Outil d'Audit Défensif du Transport pour
Applications Mobiles

Réalisé par :

Yassir KADOUARI
Marouane ISMAILI

SOFTWAREX - A REFORMATER OBLIGATOIREMENT

Le document est encore un rapport de module. Respecter le template SoftwareX : titre scientifique, auteurs/affiliations/emails, abstract en anglais, keywords, puis la table Code metadata obligatoire. Supprimer la page de garde de type rapport scolaire.

Module : Sécurité Mobile / Sécurité Web

MANQUE MAJEUR

Année Universitaire 2023-2024

Ajouter : version du code, lien GitHub permanent/tag, licence, DOI Zenodo/capsule reproductible, langage/outils, dependances, manuel, support email et version executable. Sans ces elements, SoftwareX peut rejeter le manuscrit comme non conforme.

STRUCTURE NON CONFORME SOFTWAREX

Remplacer la table des matieres et les chapitres par les sections SoftwareX : 1 Motivation and significance, 2 Software description, 3 Illustrative examples, 4 Impact, 5 Quality assurance/Validation, 6 Limitations/Future work, Conclusions, Metadata et Executable software version.

1 Introduction Générale 2

A AJOUTER Architecture et Fonctionnalités 3

Le manuscrit doit etre redige comme article logiciel, pas comme rapport de projet. Ajouter les figures, tableaux, equations, citations et preuves de reproductibilite dans le corps du texte. 3

3 Mise en Œuvre et Réalisation 4

3.1 Analyse de la Configuration Android 4

3.2 Module d'Intelligence Artificielle et Classification 4

3.3 Audit Cryptographique des Serveurs (SSLyze) 4

4 Conclusion et Recommandations 5

INTRODUCTION A RENFORCER

Traduire en anglais scientifique et positionner clairement la contribution logicielle : analyse statique APK, extraction endpoints, analyse TLS via SSLyze, classification des risques et generation de rapport. Eviter le style 'notre projet'.

Chapitre 1

Introduction Générale

Dans le contexte actuel de la sécurité des systèmes d'information, la protection des données en transit est un enjeu critique, particulièrement pour les applications mobiles. Les attaques de type "Man-in-the-Middle" (MitM) exploitent les faiblesses de configuration des flux réseau pour intercepter ou altérer des données sensibles.

L'objectif de ce projet est de concevoir et développer un outil professionnel nommé **Network/TLS Posture Analyzer**. Cet outil vise à automatiser l'audit défensif du transport réseau d'une application mobile, en se basant sur les recommandations académiques et industrielles.

Plutôt que de se focaliser sur des techniques de contournement offensives, notre solution adopte une approche défensive ("Blue Team"). Elle analyse la posture de sécurité d'une application de bout en bout : depuis l'analyse statique du fichier d'installation (APK), en passant par l'analyse dynamique des flux interceptés (logs proxy), jusqu'à l'audit cryptographique des serveurs de destination (TLS).

MANQUE POUR SOFTWAREX

Ajouter une comparaison argumentée avec MobSF, Androguard, APKLeaks, SSLyze, testssl.sh et outils proxy. Préciser la nouveauté : workflow intégrée APK + logs proxy + scan TLS + synthèse défensive. Citer OWASP MASVS/MASTG, Android Network Security Config, RFC TLS/HSTS et documentation SSLyze.

Chapitre 2

Architecture et Fonctionnalités

2.1 Objectifs Fonctionnels

Le cahier des charges de notre projet impose la mise en œuvre de trois axes majeurs d'analyse :

1. **Analyse Statique (APK) :** Extraction des configurations de sécurité réseau d'Android (`network_security_config.xml`), vérification de l'autorisation du trafic en clair (`cleartextTrafficPermitted`) et détection de la mise en place du *Certificate Pinning*.
2. **Inventaire des Endpoints :** Découverte des adresses IP et noms de domaines avec lesquels l'application communique, en extrayant ces données depuis l'APK ou en important des logs de proxy (Burp Suite, Wireshark).
3. **Vérification de la Posture TLS :** Scan automatisé des serveurs découverts pour vérifier la présence de protocoles obsolètes (SSLv3, TLS 1.0) et la configuration du HSTS.
4. **Intelligence Artificielle :** Intégration d'un moteur permettant de regrouper les endpoints (Production, Développement, Tierce partie, Suspects) et de générer un résumé des risques.

2.2 Architecture Technique

Notre application repose sur une architecture moderne séparant le client et le serveur :

- **Backend (Python / FastAPI) :** Gère le traitement lourd. Il intègre `pyaxmlparser` pour décompiler et lire les fichiers binaires Android, la bibliothèque `sslyze` pour l'audit cryptographique TLS, et un moteur heuristique simulant le comportement d'un LLM pour l'analyse des risques.
- **Frontend (React / Vite) :** Fournit une interface utilisateur riche et réactive (Single Page Application). Conçue avec un design *Clasemorphism* et *Dark Mode*.

CLARIFIER LE MODULE IA : le glisser-déposer (Drag & Drop) de fichiers et la génération de rapports
Si le module est heuristique, ne pas l'appeler Intelligence Artificielle sans modele, donnees, prompt, API ou validation. Decrire l'algorithme de classification, les regles, les entrees/sorties et les limites. Ajouter une table des fonctionnalites avec composants logiciels et dependances.

Chapitre 3

Mise en Œuvre et Réalisation

3.1 Analyse de la Configuration Android

L'un des vecteurs d'attaque principaux sur mobile est la mauvaise configuration du fichier `network_security_config.xml`. Lors de l'upload de l'APK, notre backend décompresse l'archive et analyse les balises XML. Le système lève une alerte critique si l'attribut `cleartextTrafficPermitted="true"` est détecté, car cela signifie que l'application accepte d'émettre des requêtes HTTP non chiffrées, exposant potentiellement les identifiants de l'utilisateur.

3.2 Module d'Intelligence Artificielle et Classification

Afin d'aider l'auditeur à prioriser ses recherches, nous avons implémenté un moteur d'analyse sémantique. Face à des centaines de requêtes proxy, ce moteur classe les URL en quatre catégories :

- **Production API** : Les serveurs légitimes de l'application.
- **Test / Dev / Staging** : Détection de serveurs de développement oubliés dans le build de production (qui ont souvent des mesures de sécurité affaiblies).
- **Third-Party (Analytics)** : Outils de télémétrie (Google Analytics, AppsFlyer) posant de potentielles questions de conformité RGPD.
- **Anomalies / Suspects** : Adresses IP directes ou domaines exotiques (.ru, .xyz)

VALIDATION OBLIGATOIRE : Vérifier la présence d'un malware ou d'un SDK compromis.

Ajouter une section Quality assurance / Validation : tests unitaires, tests d'integration, APK publics ou applications de démonstration, logs Burp/Wireshark, comparaison avec SSLyze seul, mesures temps d'analyse, precision de detection cleartext/pinning/endpoints, et matrice de resultats. Les recommandations doivent etre reliees aux preuves observees.

3.3 Audit Cryptographique des Serveurs (SSLyze)

Une fois les points de terminaison (endpoints) identifiés, le moteur lance des requêtes spécifiques (ScanCommands) pour évaluer la configuration TLS du serveur distant. Il vérifie notamment la vulnérabilité à des attaques connues comme *Heartbleed* ou *ROBOT*, ainsi que le support des mécanismes modernes comme le *Strict-Transport-Security (HSTS)* et le *Fallback SCSV*.

CONCLUSION A TRANSFORMER

La conclusion doit rappeler les artefacts logiciels livrables : repository, version, licence, capsule Zenodo, script de reproduction, interface, API, rapport PDF et cas d'utilisation. Eviter les formulations de rapport d'étudiants.

Chapitre 4

Conclusion et Recommandations

Ce projet nous a permis d'appréhender de manière concrète les défis de la sécurité du transport de données dans l'écosystème mobile. L'outil **Network/TLS Posture Analyzer** que nous avons développé constitue une solution robuste pour réaliser des audits défensifs rapides et efficaces. L'interface Web intuitive permet à un analyste cybersécurité de repérer en quelques secondes si une application a oublié de mettre en place du Certificate Pinning, si elle communique avec des domaines malveillants, ou si son infrastructure serveur est vulnérable.

Recommandations standard issues de nos audits :

1. Appliquer explicitement `cleartextTrafficPermitted="false"` dans le manifeste Android.
2. Mettre en œuvre le *Certificate Pinning* (via l'attribut `<pin-set>`) sur les endpoints critiques pour contrer les attaques MitM sophistiquées.
3. S'assurer que les serveurs Backend rejettent catégoriquement les connexions en TLS 1.0 et 1.1 et déploient l'en-tête HSTS.

REFERENCES ET LIMITES MANQUANTES

Ajouter une bibliographie complète : SoftwareX Guide for Authors, OWASP MASVS/MASTG, Android security config, SSLyze, HSTS/RFC, MobSF/Androguard, TLS best practices. Ajouter aussi une section Limitations : analyse statique limitée, dépendance réseau pour scans TLS, faux positifs sur endpoints tiers, limites du pinning, scope défensif uniquement. Respecter exactement le template SoftwareX.